Programming Fundamentals

- Lecturer: Dr. Arnold Meijster
- e-mail: a.meijster@rug.nl
- room: 5161.343 (Bernoulliborg)



Organisational matters

- The course "*Programming Fundamentals*" consists of a series of lectures, tutorials, and computer lab sessions.
- Small programs and aspects of the programming languages C and Dafny are presented in the lectures.
  - In the first weeks, we will start with programming in C.
- Students make exercises with laptops and pen(cil) and paper during the tutorials
  - So, bring a laptop with C (and later Dafny) installed!
- Students make exercises on their own laptops (BYOD) during the computer lab sessions.

Groups + Schedule

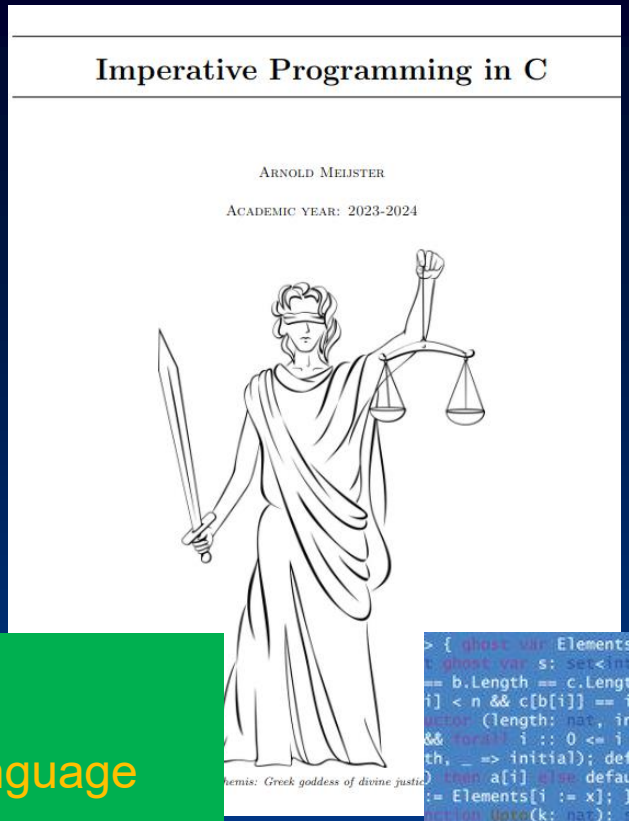
The groups + tutorial/lab schedules can be found via the following URL:

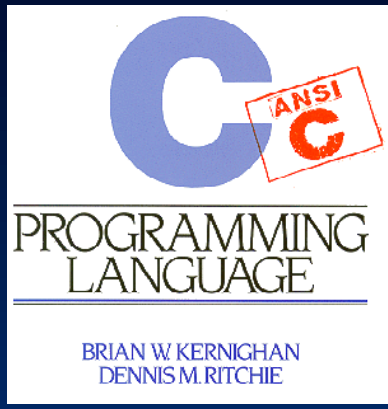
https://rooster.rug.nl/#/en/current/schedule/course-WBCS046-10

It is your own responsibility that you are at the right tutorial/lab session at the right time and place! If you do not know in which group you are, find out quickly!

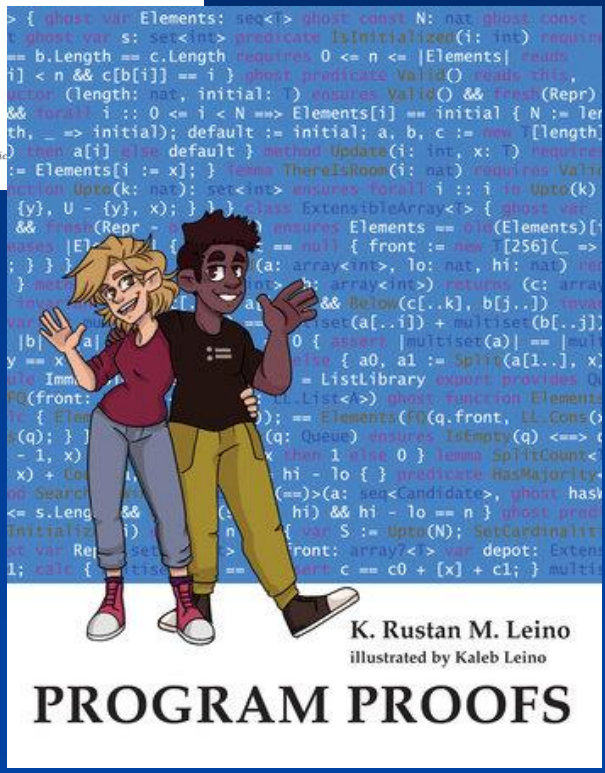
You are not allowed to switch to another group!

Literature





- Required book: Program Proofs
- Advised book: C Programming Language (ANSI C)
- Reader: Imperative programming in C
  - Pdf is available via Brightspace
- Lab sessions: weekly via Brightspace

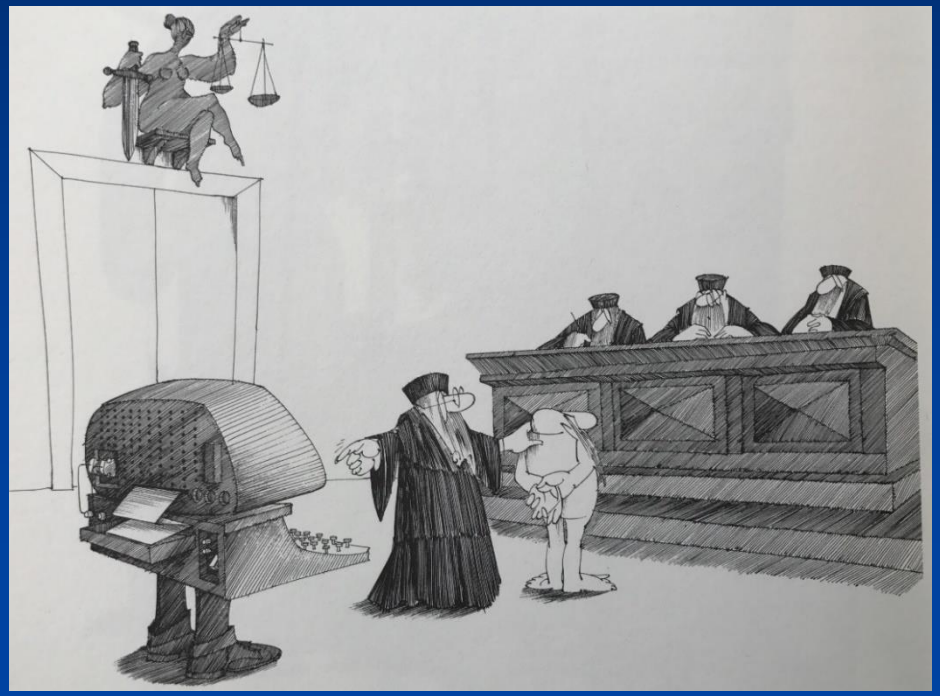


Practicals

- The lab exercises are posted on Brightspace (weekly).
  - Each lab session consists of a number of programming exercises.
- Practicals are made individually!
- There is assistance by the teaching assistants (TAs) during the scheduled lab sessions!

Practicals

- You submit your solutions (source code) to the online electronic verification system Themis. Themis is available 24/7.
  - http://themis.housing.rug.nl
  - Once a program has been accepted by Themis, the assignment is done.
  - Not accepted solutions are considered ‘not submitted’!
  - Deadlines : hard! Once second late means you failed the deadline!



Grading of Practicals

- Your submissions are initially graded by Themis.
- However, the grade might change as a result of manual assessment performed by the Teaching Assistants (TAs). They will consider criteria like programming style, layout, efficiency, etc.
- In case Themis does not accept a solution, you can submit an improved version. However, points are subtracted for too many trials:
  - ≤5 submissions: no points are subtracted (for the given problem)
  - 6-15 submissions: 25% of the points is subtracted (for the given problem)
  - > 15 submissions: 50% of the points is subtracted (for the given problem)
- So, test your programs well before submitting them!

  Note: In the first lab session, the programs will be judged manually by the TAs, but no points are subtracted (only feedback is given). However, points will be subtracted for too many submissions.

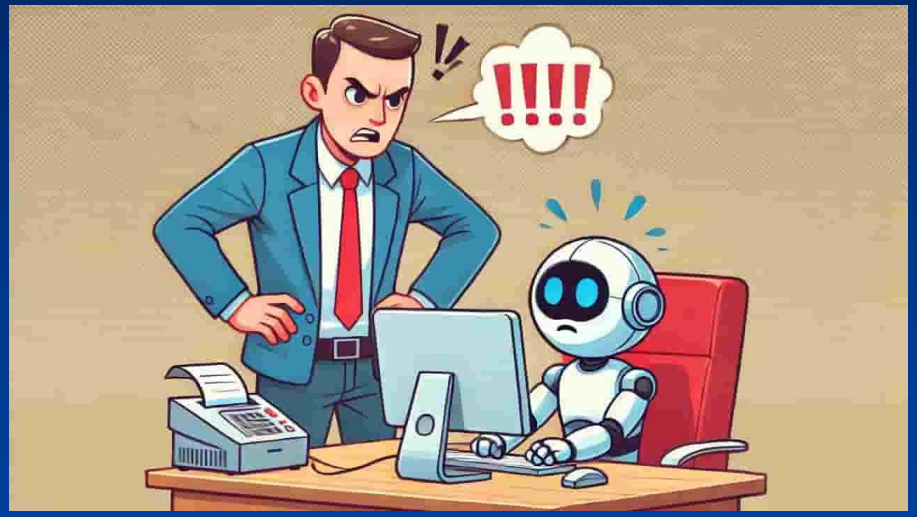
Warning: NO plagiarism!

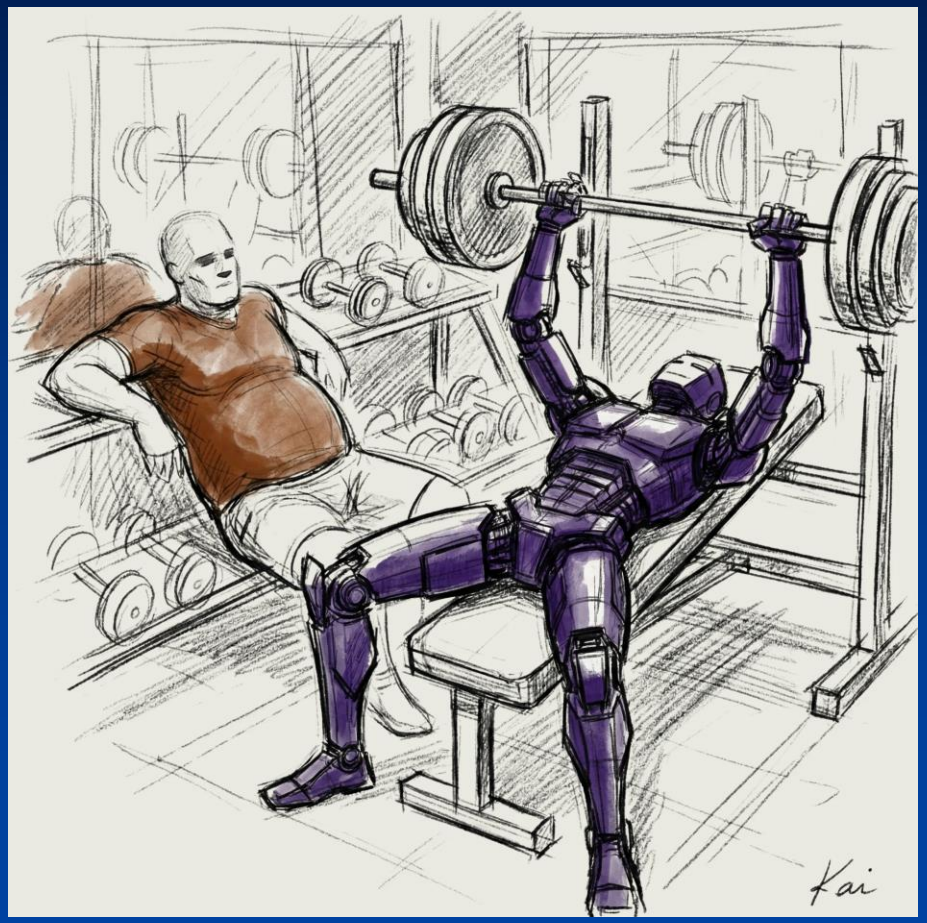




We compare submissions! If we suspect plagiarism (copying) then we forward the submissions to the board of examiners. We also check for copying/using code from internet fora.

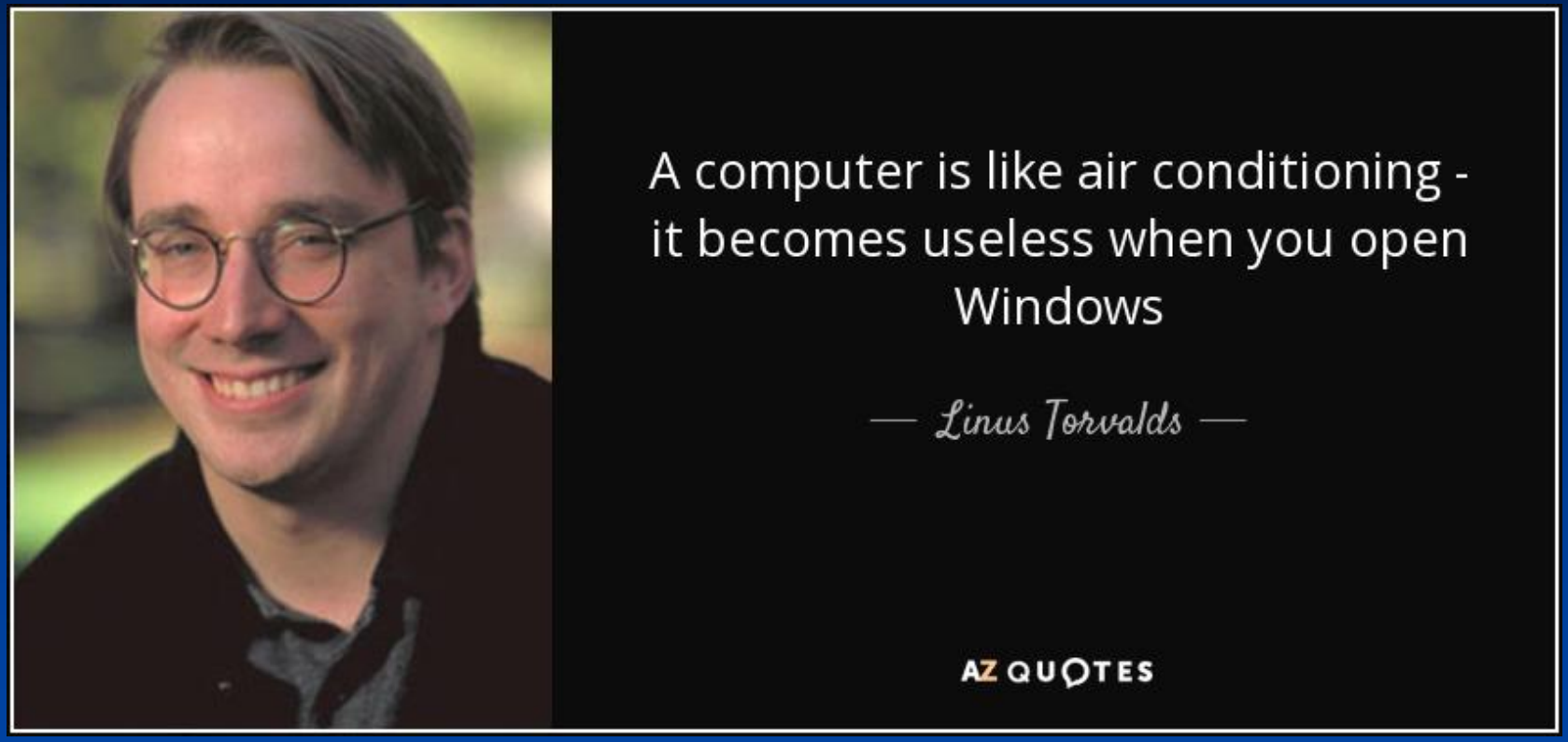
Warning: NO chatGPT!
(or any other form of generative AI)





Practicals

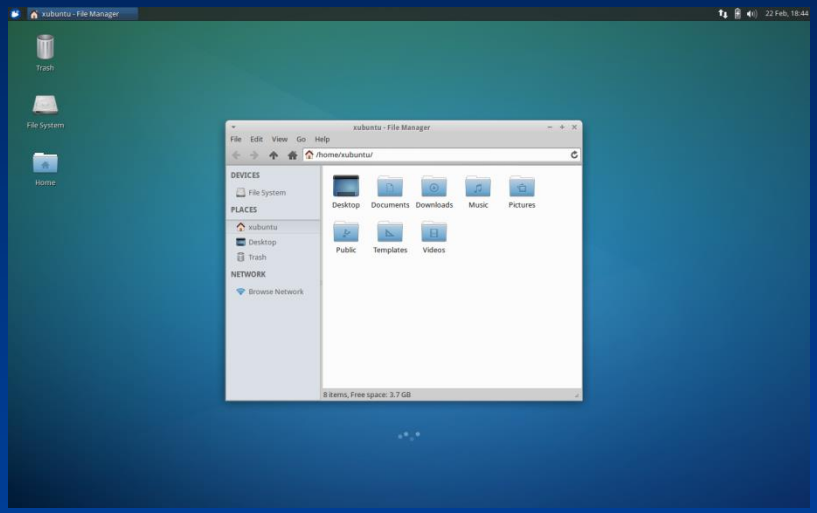
We use Linux!



Linux on your PC/Laptop ?





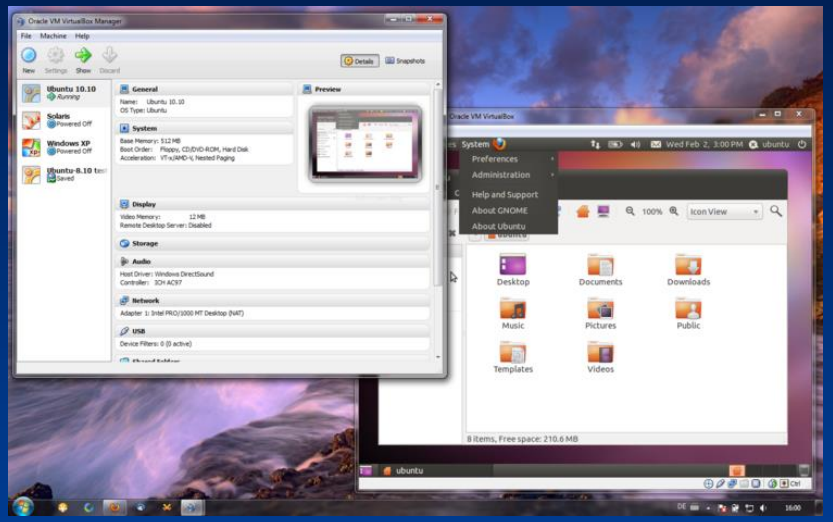


https://xubuntu.org/

Linux on your PC/Laptop ?

Replacing your windows is too scary? Use virtualbox!

https://www.virtualbox.org/



On Themis, you can find a link to a virtual disc image containing a pre-installed xubuntu 26.04 LTS system.

Username: vboxuser (password: password)

Warning: this file is BIG (18 GB zip file, 32 GB after unzipping)!

Helpdesk

Outside the scheduled hours for the lab sessions the e-mail address **`progfunrug@gmail.com`** is available as a digital helpdesk.





Examination / Grading

- The grade P for the practical sessions is the mean of the grades for the lab sessions.
  - P needs to be a 5 or higher to pass the course.
  - If a session is missing, then you will receive the grade 1 for that session.
  - There are no resits for lab assignments!

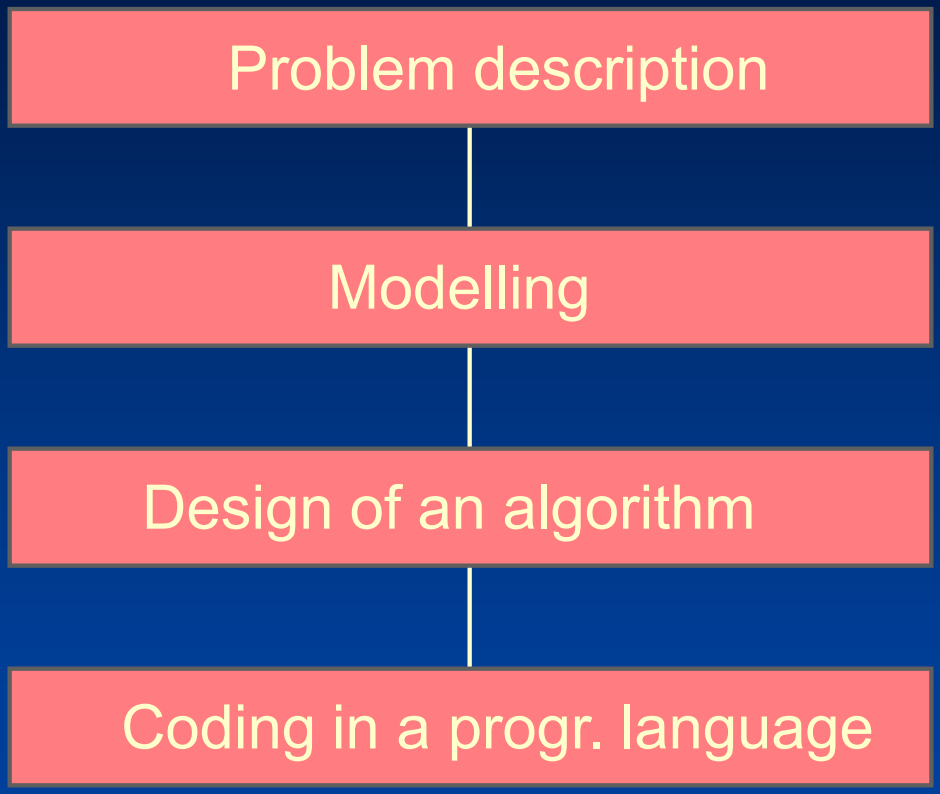
- The grade E for the examination part of your grade is the mean of the grades for 3 sub-examinations.
  - See schedules for the dates!
  - E needs to be a 5 or higher to pass the course.
  - If you missed a sub-examination, then you will receive the grade 1 for that sub-examination.
  - For each sub-examination, there is a resit. In case of a resit, the resit grade replaces the old grade.

- Final Grade: $F = (P + E)/2$ provided that $E, P \geq 5$ (otherwise $F = \min(P, E)$)

- To pass the course, you need $F \geq 5.5$

Programming

Programming is more than coding:



During this course, we will pay attention to all these aspects of programming.

Palindrome (problem description)

Task: write a program that decides whether a word is a *palindrome* or not.

A **palindrome** is a word that reads the same in either forward or reverse direction.

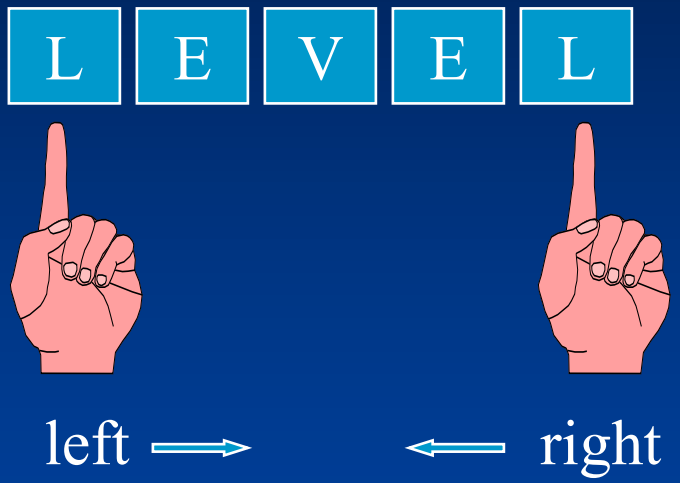
- civic
- level
- madam
- radar
- racecar
- meetsysteem (Dutch)

Palindrome (modelling)

We regard a word as a sequence of characters.

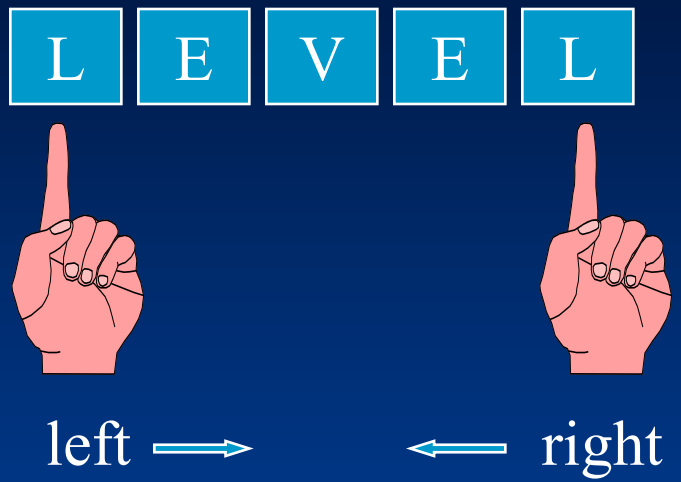
We can use two fingers as pointers in the word:

- one from head to tail
- one from tail to head.



A word is not a palindrome if the letter at location **`left`** differs from the letter at location **`right`** (for **`left ≤ right`**).

Palindrome (algorithm design)



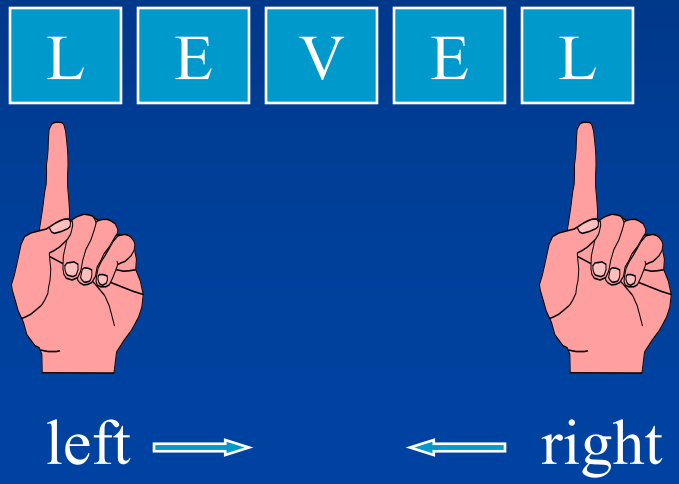
If both fingers point to the same character, then we go on.

Otherwise, we stop: the word is not a palindrome.

We can also stop if the fingers point to the same position (or even passed each other): the word is a palindrome.

Palindrome (algorithm design)

```
LET left point to the first character
LET right point to the last character
REPEAT
   IF character at left equals character at right
   THEN
    shift left 1 position to the right
    shift right 1 position to the left
   ELSE
    OUTPUT "NO"
    STOP
UNTIL left >= right
OUTPUT "YES"
STOP
```



Palindrome (algorithm design)

There are other solutions. As a teaser, here is another idea:

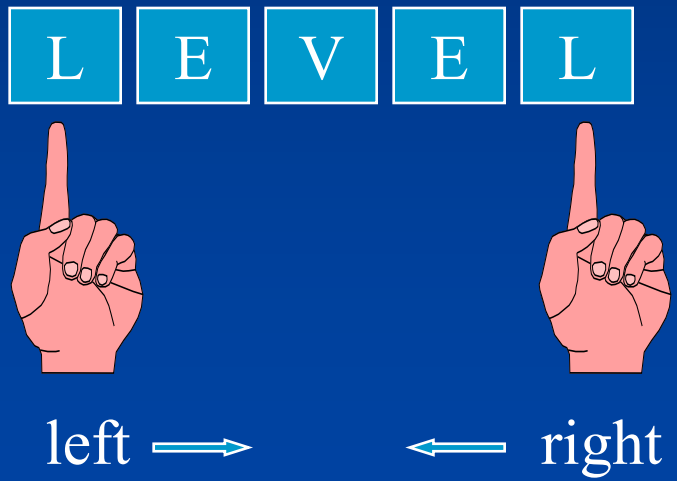
- An ‘empty’ sequence of characters is a palindrome.
- A single character is a palindrome.
- A sequence of characters that has the form cXd is not a palindrome (provided that c and d are not the same).
- A sequence of characters that has the form cXc is a palindrome if X is a palindrome. (recursion!)

Palindrome (coding)

- Finally, we code our algorithm in some programming language. We will use the programming language C.
- This may seem to be the real hard part, but it really is not. Once you have designed an algorithm, the hard part is done!
- Designing good algorithms can be hard. Converting them into C code is only a matter of routine.

Palindrome (coding)

```
int left = 0;
int right = length - 1;
while ((left < right) && (word[left] == word[right])) {
  left++;
  right--;
}
if (left >= right) {
  printf("YES\n");
} else {
  printf("NO\n");
}
```



Compiling a C program (Linux)



- Editor: visual studio `code` (or `geany` , or `emacs`, or `vi`, or ....)
- Compiler: `gcc`
- Compilation: `gcc -std=c99 -Wall progr.c`
  - Converts C source code in a binary executable `a.out`
- Execution: `./a.out`

Statements

- A program is a sequence of *statements*.
- A statement is some elementary command:
  - print “Hello world” on the screen
  - Increment the value of some variable
  - Repeat something 10 times
  - Etc.
- Statements are separated by a semicolon `;`
- Convention: one statement per line

Comments

- Comments in a program are human-readable text
  - Enclosed by **/*** and ***/**
    - May span several lines of code
  - Or, single line comments starting with **//**
- Comments are ignored by the compiler
  - Intended to make the logic of a program easier to understand
- Don’t be lazy! Use comments!
- Example:

```
/* This is a multi line
 * comment.
 */

// This is a single line comment.
```

Structure convention

```
/* file    : skeleton.c */
/* author : Joe Sixpack (joe@student.rug.nl) */
/* date    : Mon Sep 6 2026 */
/* version: 1.0 */

/* Description:
 * Here should be a description of
 * what this program is supposed to do.
 */

#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
  /* here your own code */
  return 0;
}
```

First example: hello world

```
/* file    : hello.c */
/* author : Joe Sixpack (joe@student.rug.nl) */
/* date    : Mon Sep 6 2026 */
/* version: 1.0 */

/* Description:
 * This is the standard hello-program.
 * It prints 'Hello world!' on the screen.
 */

#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
  printf ("Hello world!\n");
  return 0;
}
```

file: hello.c

Compilation: gcc –std=c99 –Wall hello.c

Not like this !!!!!!!!!!!!

```
/* file    : hello.c */
  /* author : Joe
 Sixpack (joe@student.rug.nl) */
/* date    : Mon Sep 8 20
26 *//* version: 1.0 */

/* Description:
 * Th
is is the standard hello-program.
 * It prints 'Hello world' on the screen.
 */

#include <stdio.h>
#include <stdlib.h>
int
main(int argc,
char *                          argv[]) { printf (
   "Hello world!\n");  return    0;}
```

Integers: operators

For the time being, we will only use the data type `int`

operation	operator	example
addition	+	17 + 5=22
subtraction	-	17 - 5=12
multiplication	*	17*5=85
integer division	/	17/5=3
remainder (modulo)	%	17%5=2

Note that `x` is equal to `d*(x/d) + x%d` (provided that `d` is non-zero).

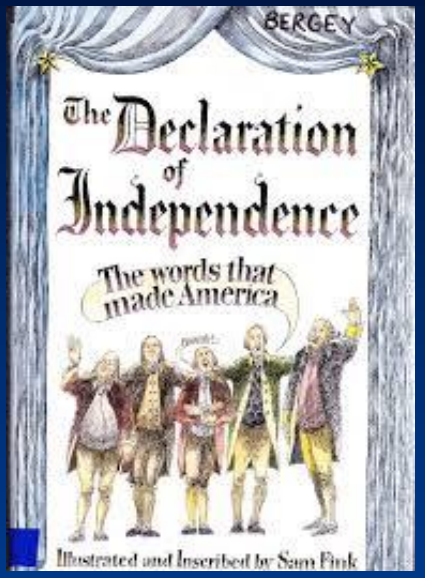
Variables

- In imperative programming *variables* play a central role
  - Data is stored in variables
  - A program modifies the values of variables

  - Examples:
    - A student number can be stored in a variable of the type **int**.

Declaration of variables

- You need to *declare* a variable before usage.
  - It tells the compiler to reserve memory space.
  - Examples:
    - `int x, small, numberOfPersons;`
    - `int independence;`



- Variable names (*identifiers*) consist of letters, digits and underscores.
  - Convention: we will use the *camelCase* convention for identifiers: each identifier start with a lower-case letter, followed by letters and digits. Each word starts with an upper-case letter (e.g. `numberOfPersons`). In this course, we do not use other characters, like underscores!
- Note: C is case sensitive! (`small` differs from `Small` or `SMALL`).

Assignment

The simplest way to give a variable a value is the *assignment statement*.

Assignments have the following form:

<variable> = <expression>;

```
int x, y;

x = 17;     /* x==17              */
y = x + 9; /* x==17, y==26 */
x = y;      /* x==26, y==26 */
```

Assignment

```
<variable> = <expression>;
```

Conditions:

- The variable has been declared,
- **<variable>** and **<expression>** have compatible types,
- the expression can be evaluated,
- the value of the expression ‘fits’ in the variable.

Declaration with initialisation

Instead of writing

```
int x, y;
x = 17;   /* x==17       */
y = 12;   /* x==17, y==12 */
```

it is much shorter (and easier to read) to write

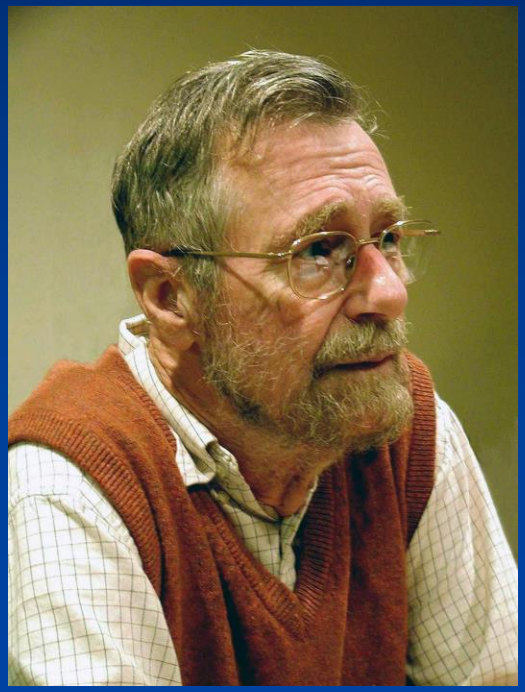
```
int x = 17, y=12;   /* x==17, y==12 */
```

Formalization and Reasoning

- Formal reasoning
  - Establish correctness for all (possibly infinite) cases
  - Know properties ahead of time
  - Requires abstract reasoning and careful thinking
  - Needs logic
- Today: develop formal reasoning for assignments
  - What is true about a program's state as it executes a series of assignments?
  - Later in the course (Dafny part): more complicated constructs, like loops.

Why Reason About Programs?

"**Program testing** can be used to show the presence of bugs, but never to show their absence!" — Edsger W. Dijkstra



Hoare Triples

A *Hoare triple* is code wrapped in two assertions

{ P } S { Q }

P is the precondition

S is the code (statement)

Q is the postcondition

Hoare triple **{P} S {Q}** is *valid* if:

For all states where **P** holds, executing **S** always produces a state where **Q** holds

Otherwise the triple is *invalid*

Hoare Triple Examples

Valid or invalid?

Assume all variables are integers without overflow

```
{x != 0} y = x*x; {y > 0}
```
valid

```
{z != 1} y = z*z; {y != z}
```
invalid

```
{x >= 0} y = 2*x; {y > x}
```
invalid

```
{true} x = y; z = x; {y==z}
```
valid

Assignments

Exercise: find a series of assignments that satisfies the following *specification*:

```
/* x == A && y == B */
???
/* x == A + B && y == A*B */
```

Note: **A** and **B** are so-called *specification constants* (written in upper case letters) and not variables!
The program should work for any possible value for **A** and **B**.

Assignments

```
/* x == A && y == B */
/* x + y == A + B && y == B */
x = x + y;
/* x == A + B && y == B */
/* x == A + B && (x-y)*y == A*B */
y = (x-y)*y;
/* x == A + B && y == A*B */
```

Assignments

Exercise: find a series of assignments that satisfies the following *specification*:

```
/* x == A && y == B */
???
/* x == B && y == A */
```

Assignments (swap)

```
/* x == A && y == B */
z = x;
/* x == A && y == B && z == A */
x = y;
/* x == B && y == B && z == A */
y = z;
/* x == B && y == A && z == A */
/* x == B && y == A */
```

Assignments (tricky swap)

```
/* x == A && y == B */
/* x + y == A + B && y == B */
x = x + y;
/* x == A + B && y == B */
/* x == A + B && x - y == A */
y = x - y;
/* x == A + B && y == A */
/* x - y == B && y == A */
x = x - y;
/* x == B && y == A */
```

Standard Input/Output (I/O)

- `#include <stdio.h>`
- ***Standard output*** is the screen
- ***Standard input*** is the keyboard

Formatted Output: printf

- General form: `printf("format", …)`

- Examples:

```
printf("Hello world\n");

printf("%d", 123);

printf("%d + %d = %d\n", 20, 22, 20+22);
```

Formatted Input: scanf

General form:

```
n = scanf("format", &var, …)
```



BEWARE &

Examples:

```
int a, x, y, n;

scanf("%d", &a);    /* beware!: & */
printf("a==%d\n", a);
n = scanf("%d %d", &x, &y);
printf("x==%d, and y==%d\n", x, y);
printf ("You typed in %d numbers\n", n);
```

Formatted Input: scanf

`scanf()` returns the number of matching input elements

For example: `n = scanf("%d %d", &x, &y);`

Returns (value of `n`):

- 2 two numbers read from the keyboard
- 1 only one number was read from the keyboard
- 0 no match (i.e. no numbers on the input)
- `EOF` End of file (CTRL-D)

A simple calculator

Exercise: write a program that reads two integers from the keyboard and then prints the sum, difference and product of these numbers.

A simple calculator

file: calculator.c

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
   int x, y;
   printf ("Type 2 integers: ");
   scanf("%d %d", &x, &y);

   printf("%d + %d = %d\n", x, y, x + y);
   printf("%d - %d = %d\n", x, y, x - y);
   printf("%d*%d = %d\n", x, y, x*y);
   return 0;
}
```

Special assignments

In C, there are some special short hand notations available for frequently occurring assignments.

assignment:

```
x = x + 1;
x = x - 1;
x = x op exp;
   e.g. x = x*(y + 1);
```

Short hand notation:

```
x++;
x--;
x op= exp;
   e.g. x *= y + 1;
```

Special assignments

The assignment `x++` is itself a value that can be assigned to some variable:

```
int x=41, y;
y = x++;
printf("x=%d, y=%d\n", x, y);
```

```
// equivalent:
int x=41, y;
y = x;
x = x + 1;
```

The output of this program fragment will be:

`x=42, y=41`

Special assignments

Besides `x++` (post-increment) we can also write `++x` (pre-increment):

```
int x=41, y;
y = ++x;
printf("x=%d, y=%d\n");
```

```
// equivalent:
int x=41, y;
x = x + 1;
y = x;
```

The output of this program fragment will be:

```
x=42, y=42
```

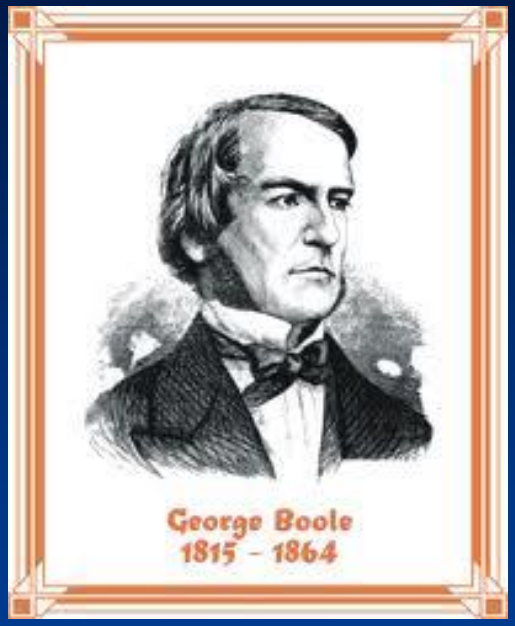
Special assignments

The same holds for `x--` (post-decrement) and `--x` (pre-decrement):

```
int x=41, y;
y = x--;
// x==40, y==41
```

```
int x=41, y;
y = --x;
// x==40, y==40
```

Boolean algebra



Boolean algebra acts on the thruth values TRUE (1) and FALSE (0) only.



C-convention: 0 represents false. Any other non-zero value represents true.

Boolean operators

`!`	not (negation)
`&&`	and (conditional conjunction)
`||`	or (conditional disjunction)

Boolean operators

0=false, 1=true

AND (&&)

Inputs		Output
0	0	0
0	1	0
1	0	0
1	1	1

OR (||)

Inputs		Output
0	0	0
0	1	1
1	0	1
1	1	1

NOT (!)

Input	Output
0	1
1	0

Comparison operators

==	equal
!=	not equal
<	less than
<=	less than or equal
>	greater than
>=	greater than or equal

Examples: Boolean expressions

```
int x = 123, y = 200;

int a = x < y;

/* a == 1 (TRUE) since 123 < 200 */


int doorIsLocked, lightsOn, readyToGo;

readyToGo = doorIsLocked && !lightsOn;



int canBuy, hasCash, hasCreditCard, money, cost;

canBuy=(hasCash && money>=cost) || hasCreditCard;
```

IF-statement (conditional statement)

General form:

```
if (condition) {
    thenStatements;
};
```

IF-ELSE-statement

General form:

```
if (condition) {
    thenStatements;
} else {
    elseStatements;
};
```

BEWARE!

- The following two if-statements are NOT the same!

```
if (1 < i  &&  i < 10) {
   printf("1 < i < 10\n");
}
```

```
if (1 < i < 10) {
   printf("1 < i < 10\n");
}
/* 1 < i < 10 means (1<i)<10, which is always true! */
```

- It is better to write (use parentheses!):

```
if ((1 < i) && (i < 10)) {
   printf("1 < i < 10\n");
}
```

IF-statement

Exercise: read an integer from the keyboard and print whether the number is even or odd.

```
int number;

printf("Type an integer: ");
scanf("%d", &number);

if (2*(number/2) == number) {
  printf("%d is even.\n", number);
} else {
  printf("%d is odd.\n", number);
};
```

IF-statement (better)

Exercise: read an integer from the keyboard and print whether the number is even or odd.

```
int number;

printf("Type an integer: ");
scanf("%d", &number);

if (number%2 == 0) {
  printf("%d is even.\n", number);
} else {
  printf("%d is odd.\n", number);
};
```

IF-statement (alternative)

Exercise: read an integer from the keyboard and print whether the number is even or odd.

```
int number;

printf("Type an integer: ");
scanf("%d", &number);

if (number%2) {
  printf("%d is odd.\n", number);
} else {
  printf("%d is even.\n", number);
};
```

Short circuit evaluation

In C we have short circuit Boolean evaluation:

Conditional AND: P && Q: IF P is true THEN Q ELSE false

Conditional OR: P || Q: IF P is true THEN true ELSE Q

```
if ((d!=0) && (x%d==0)) {
  printf("%d divides %d\n", d, x);
}
```

```
if ((x%d==0) && (d!=0)) {
  printf("%d divides %d\n", d, x);
}
```

Conditional/ternary expression

- General from:

```
condition ? expr1 : expr2
```

- Example:

```
absValue = (n < 0 ? -n : n);
```

- Much better (shorter) than:

```
if (n < 0) {
  absValue = -n;
} else {
  absValue = n;
}
```

Conditional/ternary expression

- The ternary expression can also be quite useful in combination with printf:

```
if (number%2 == 0) {
  printf("even\n");
} else {
  printf("odd\n");
};
```

```
// equivalent statement
printf(number%2 == 0 ? "even\n" : "odd\n");
```

Prime number < 100?

- Exercise: read an integer x from the keyboard (where 1 < x < 100), and print whether the number is prime or not.

Prime number < 100?

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
  int x;
  printf("Type an integer x (1 < x < 100): ");
  scanf("%d", &x);
  if ((x <= 1) || (x > 99)) {
    printf ("Hey, I told you: 1 < x < 100!\n");
    return -1;  /* stop program with error code -1 */
  }

  if ((x==2) || (x==3) || (x==5) || (x==7)) {
    printf ("%d is a prime number.\n", x);
    return 0;
  }
  if ((x%2 != 0) && (x%3 != 0) && (x%5 != 0) && (x%7 != 0)) {
      printf ("%d is a prime number.\n", x);
  } else {
      printf ("%d is not a prime number.\n", x);
  }
  return 0;
}
```

Prime number < 100?

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
  int x;
  printf("Type an integer x (1 < x < 100): ");
  scanf("%d", &x);
  if ((x <= 1) || (x > 99)) {
    printf ("Hey, I told you: 1 < x < 100!\n");
    return -1;  /* stop program with error code -1 */
  }

  if ((x==2) || (x==3) || (x==5) || (x==7)) {
    printf ("%d is a prime number.\n", x);
    return 0;
  }
  if ((x%2) && (x%3) && (x%5) && (x%7)) {
      printf ("%d is a prime number.\n", x);
  } else {
      printf ("%d is not a prime number.\n", x);
  }
  return 0;
}
```

Nested IF-statements

```
if (grade == 10) {
  printf("outstanding\n");
} else {
  if (grade == 9) {
    printf("excellent\n");
  } else {
    if (grade == 8) {
      printf("very good\n");
    } else {
      if (grade == 7) {
        printf("good\n");
      } else {
        if (grade == 6) {
          printf("satisfactory\n");
        } else {
          if (grade == 5) {
            printf("almost satisfactory\n");
          } else {
            printf("(very) unsatisfactory\n");
          }
        }
      }
    }
  }
}
```

Too much nesting!
We can do better!

Switch statement

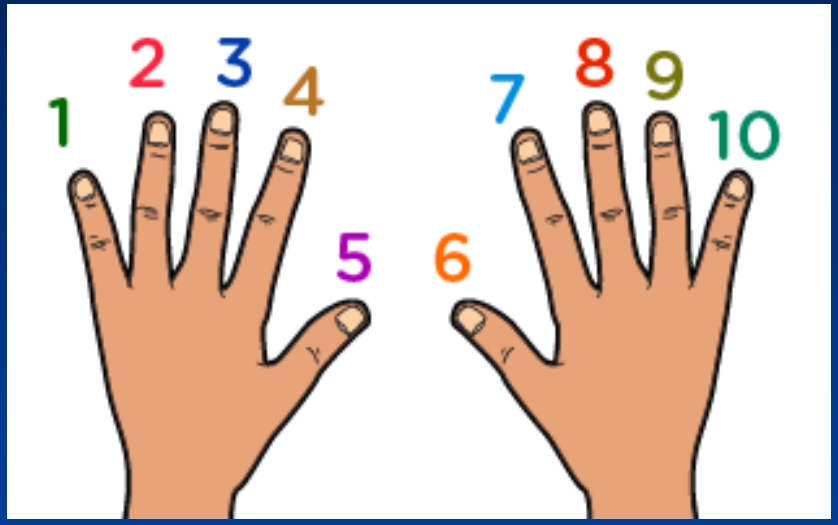
```
switch (grade) {
case 10: printf("outstanding\n");
  break;
case 9:  printf("excellent\n");
  break;
case 8:  printf("very good\n");
  break;
case 7:  printf("good\n");
  break;
case 6:  printf("satisfactory\n");
  break;
case 5:  printf("almost satisfactory\n");
  break;
default: printf("(very) unsatisfactory\n");
}
```

Switch statement (fall-through)

```
switch (grade) {
case 10: printf("outstanding\n");
  break;
case 9:  printf("excellent\n");
  break;
case 8:  printf("very ");
case 7:  printf("good\n");
  break;
case 5:  printf("almost ");
case 6:  printf("satisfactory\n");
  break;
default: printf("(very) unsatisfactory\n");
}
```

Decimal representation of integers

10 digits: 0, 1, 2, 3, 4, 5, 6, 7, 8, 9



$$156 = 1 \times 10^2 + 5 \times 10^1 + 6 \times 10^0$$

Representing integers: Base-N

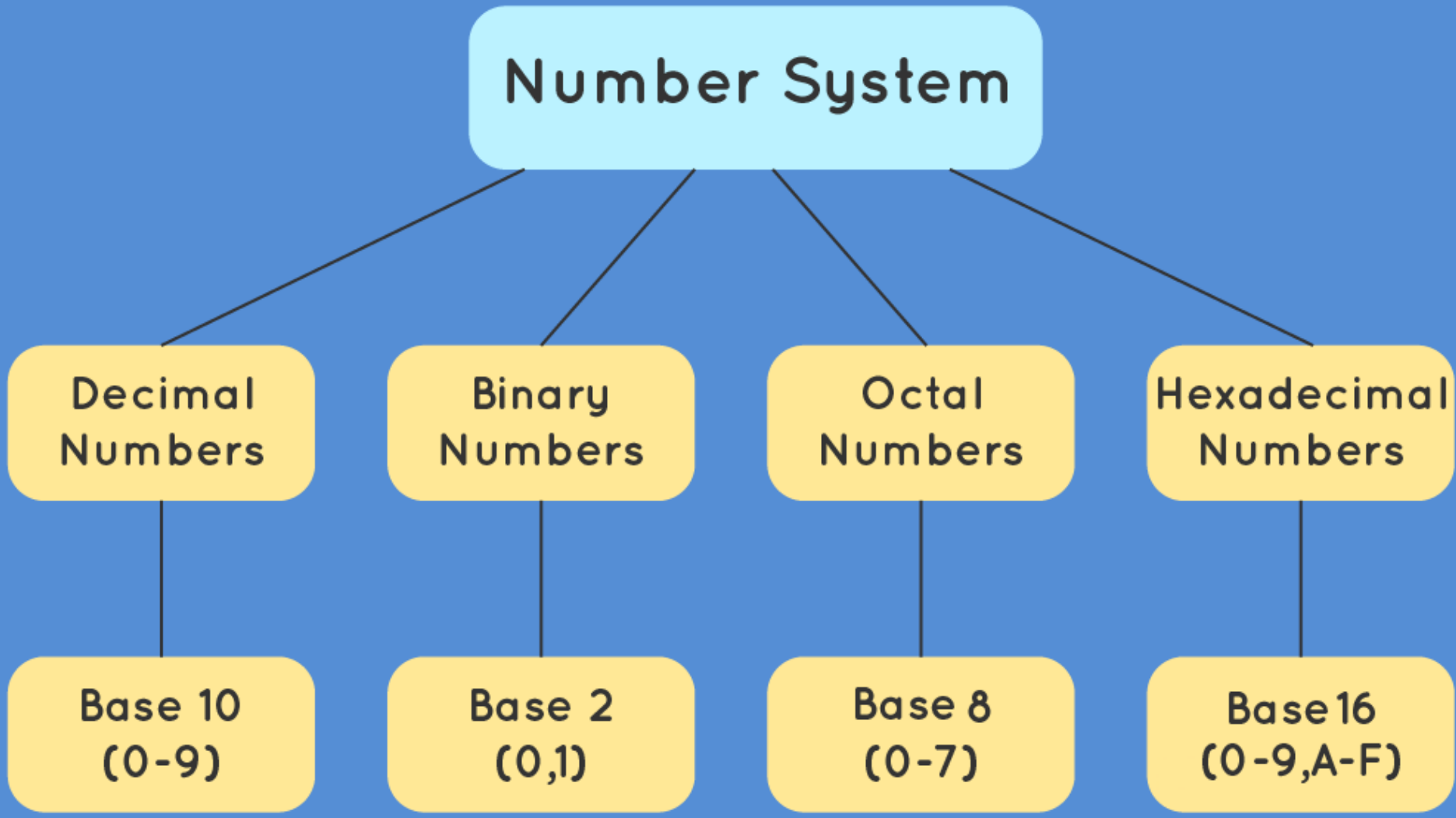
Base B:

B digits: 0, 1, 2, 3, 4, 5, …, B-1

Positional number system

$$x = d_{n-1} \cdots d_4 d_3 d_2 d_1 d_0 = \sum_{0 \le i < n} d_i \cdot B^i$$

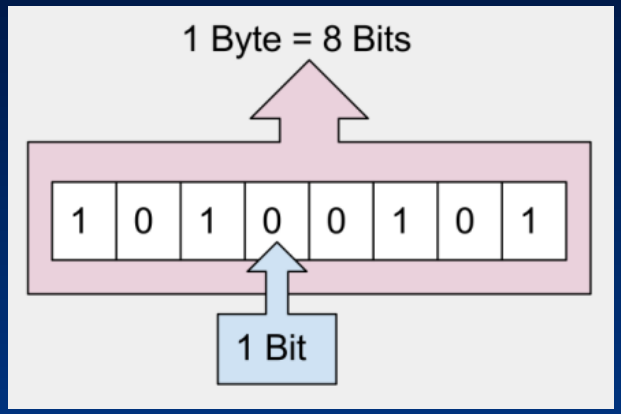
Digit d_o is the least significant digit (LSD).
Digit d_{n-1} is the most significant digit (MSD).


Number System
Decimal Numbers
Binary Numbers
Octal Numbers
Hexadecimal Numbers
Base 10 (0-9)
Base 2 (0,1)
Base 8 (0-7)
Base 16 (0-9,A-F)

Binary representation of integers

Base 2

2 bits: 0, 1



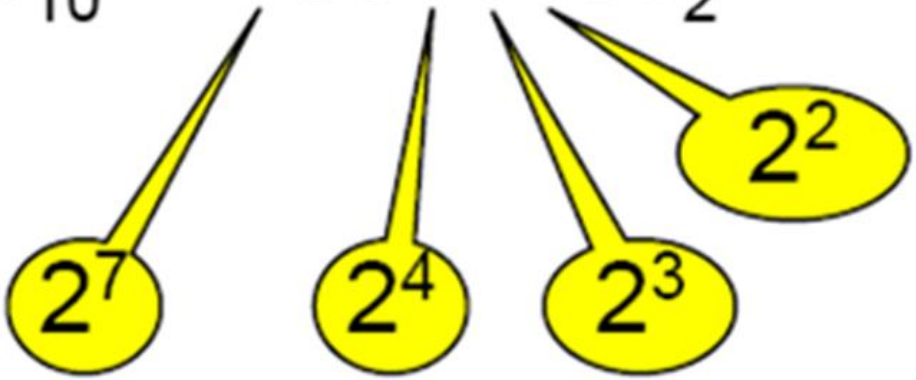
Binary digits are called *bits*

Bit b_o is the least significant bit (LSB).

Bit b_{n-1} is the most significant bit (MSB).

Binary representation of integers

$156_{10} = 10011100_2$



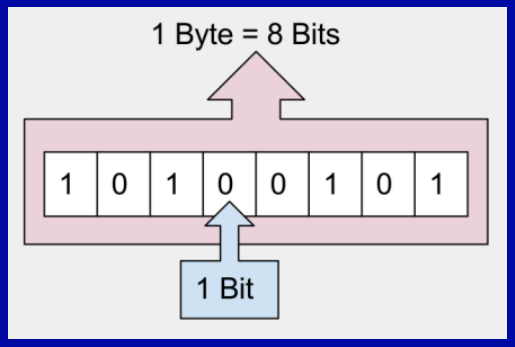
$156_{10} = 1x2^7 + 1x2^4 + 1x2^3 + 1x2^2$

$= 128 + 16 + 8 + 4$

Binary representation of integers

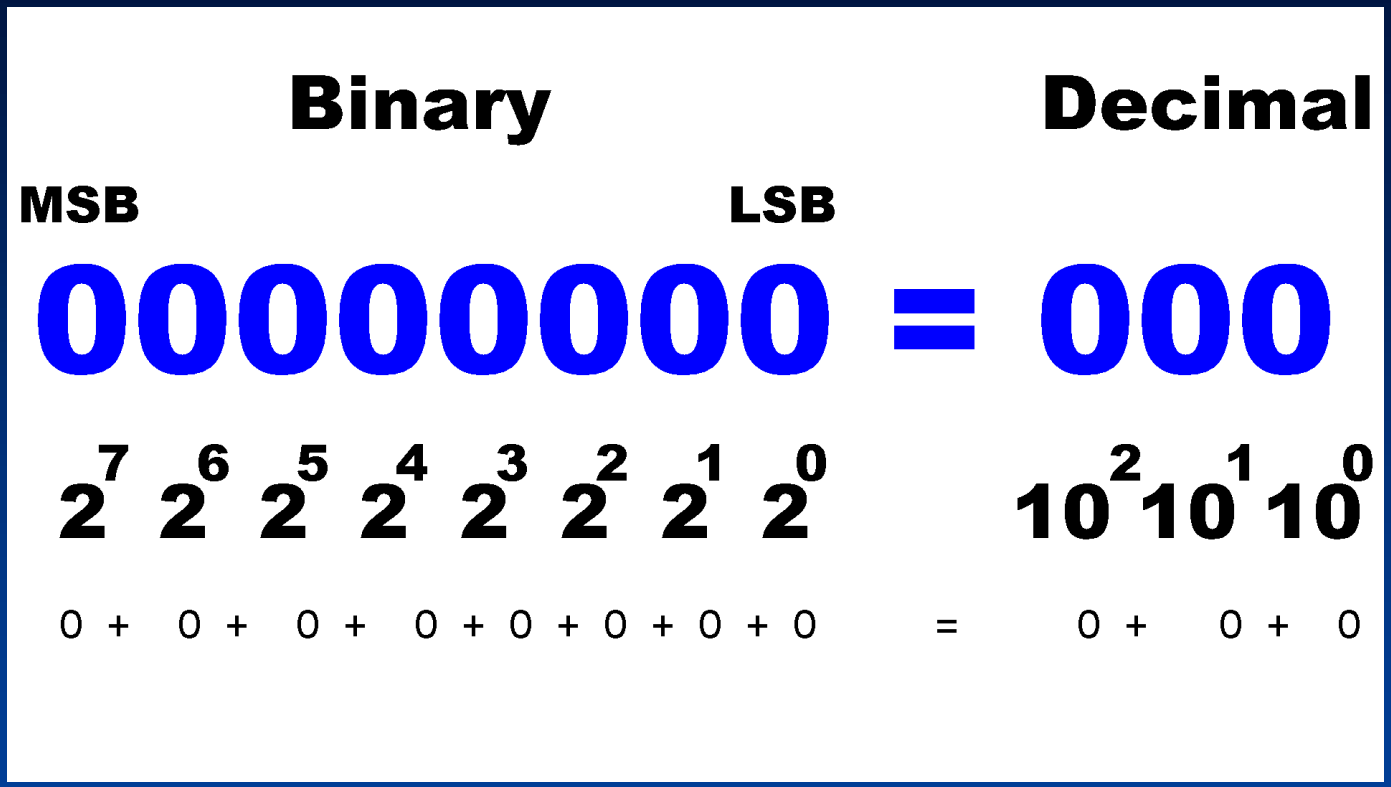
Which decimal number is represented by the binary representation 01000110?

0	1	0	0	0	1	1	0



$$= 128 + 32 + 4 + 1 = 165$$

Bit permutations (byte)



MSB = Most Significant Bit

LSB = Least Significant Bit

Using n bits we can represent 2^n numbers.
A *byte* consists of 8 bits, so we can represent 256 numbers: 0..255.

Converting Decimal to Binary

Say we want to convert 23 to binary.

We start with all bits in “off” (0) position.

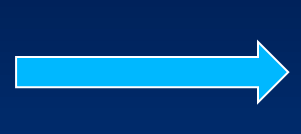
We find the largest power of two less or equal to 23 (which is $2^4 = 16$) and turn the corresponding 4th bit “on” (1). Note that the LSB bit is the 0th bit!

For the remaining bits, we compute 23-16=7 and repeat.

Converting Decimal to Binary

Converting 23 to binary byte:

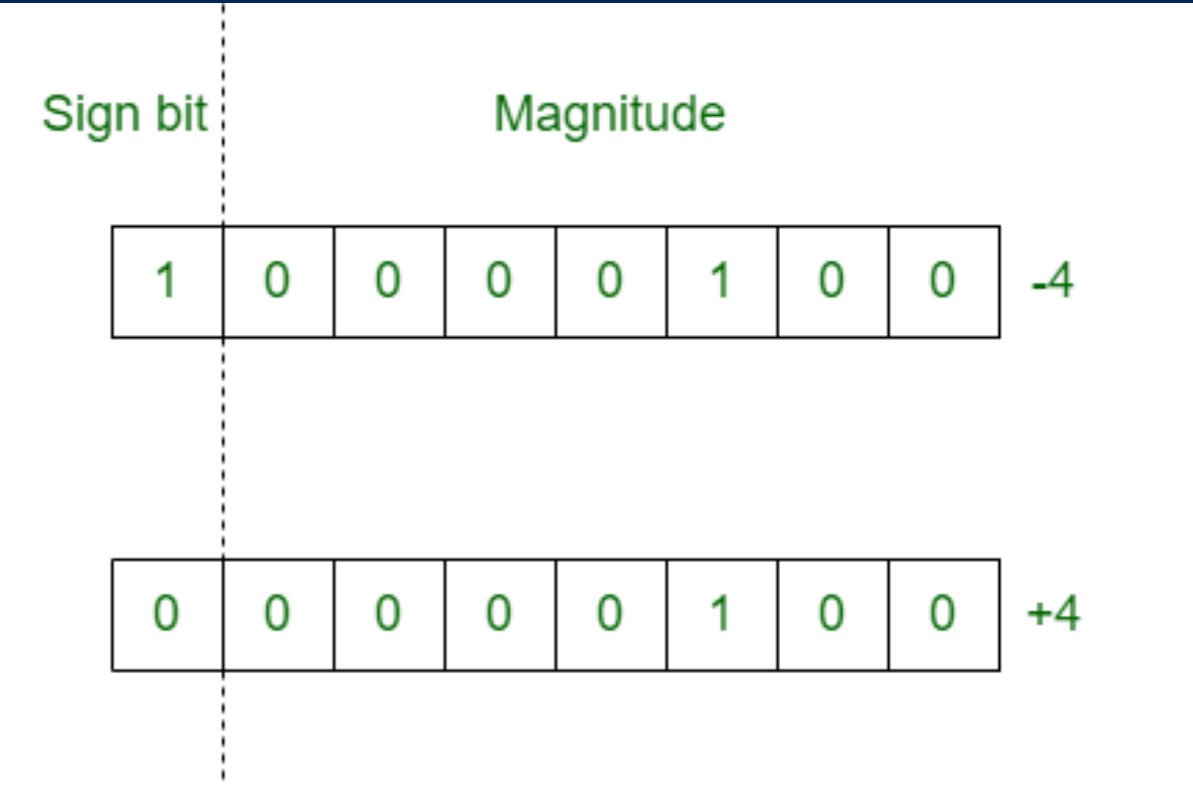
- Start with all bits “off”
- $2^4 = 16 \leq 23$: set 4th bit
- 23-16= 7
- $2^2 = 4 \leq 7$: set 2nd bit
- 7 - 4 = 3
- $2^1 = 2 \leq 3$: set 1st bit
- 3-2=1
- $2^0 = 1 \leq 1$: set 0th bit=LSB



0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	0	1	0	1	0	0
0	0	0	1	0	1	1	0
0	0	0	1	0	1	1	1

What about negative integers?

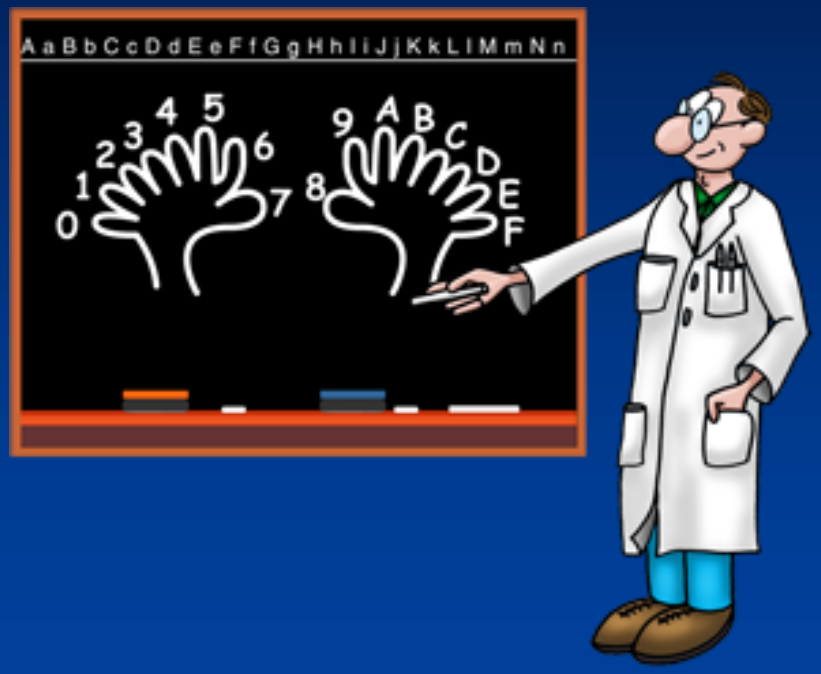
Well, at the expense of one bit (MSB) we can represent those too. For example:



Hexadecimal Number System

Base 16

Sixteen digits: 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F



Example: $9F_{16} = (9 \cdot 16^1 + 15)_{10} = 159_{10}$

Hexadecimal Number System

Base 16

Sixteen digits: 0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F

A hexadecimal digit is represented by four bits. These four bits together are called a *nibble*.

0000	0
0001	1
0010	2
0011	3

0100	4
0101	5
0110	6
0111	7

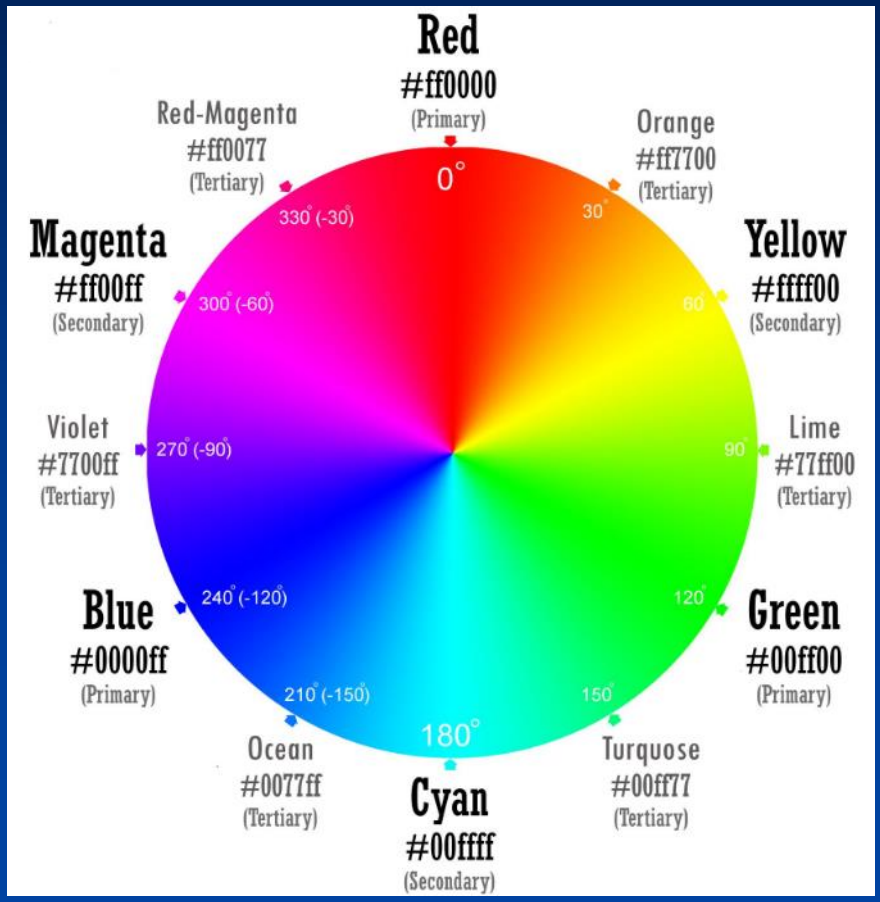
1000	8
1001	9
1010	A
1011	B

1100	C
1101	D
1110	E
1111	F

Example: $9F_{16} = (9 \cdot 16^1 + 15)_{10} = 159_{10} = 10011111_2$

Hexadecimal Number System

You may have encountered the hexadecimal number system in choosing colours in some graphics editor.



The hexadecimal number FFFF00 in the RGB(=Red Green Blue) colour system is the colour "Yellow".

Red: $FF_{16} = 255_{10}$,

Green: $FF_{16} = 255_{10}$,

Blue: $00_{16} = 0_{10}$

Octal Number System

Base 8

Eigh digits: 0,1,2,3,4,5,6,7

An octal digit is represented by three bits.

000	0
001	1
010	2
011	3

100	4
101	5
110	6
111	7

Example: $16_8 = (1 \cdot 8^1 + 6)_{10} = 14_{10} = 001110_2$

Printf in binary/hex/octal

```
int main(int argc, char *argv[]) {
  int number = 159;
  printf("decimal:     %d.\n", number);
  printf("binary:      %b.\n", number);
  printf("hexadecimal: %x.\n", number);
  printf("Hexadecimal: %X.\n", number);
  printf("octal:       %o.\n", number);
  return 0;
}
```

```
decimal:     159.
binary:      10011111.
hexadecimal: 9f.
Hexadecimal: 9F.
octal:       237.
```

Numerical types in C (on a standard PC)

type	kind	bits	maximum
short	integer	16	32767
int	integer	32	2147483647
long	integer	64	>10E18
float	floating point	32	>10E38
double	floating point	64	>10E308

For each type the minimum value is equal to –(maximum+1).

For example, the smallest `int` value is `-2147483648`.

Integer types in C (unsigned)

type	bits	maximum
unsigned short	16	65535
unsigned int	32	4294967295
unsigned long	64	18446744073709551615

For each unsigned type, the minimum is 0 (obviously).

Overflow (int)

Warning: be aware that you have limited int values!

```
int a,b,c,d;
a = 2147483646; // 2 to the power 31 - 2
b = a;
b++;                    // b == 2147483647
c = b;
c++;                    // c == -2147483648 (overflow!)
d = c;
d++;                    // c == -2147483647
```

Underflow (int)

Warning: be aware that you have limited int values!

```
int a,b,c;
a = -2147483648; // a == -2147483648
b = a;
b--;             // b == 2147483647 (underflow!)
c = b;
c--;             // c == 2147483646
```

Floating point numbers: accuracy

Warning: be aware that you have limited accuracy!

For example, do not expect that the square root of 36 is 6.
It might very well be 5.99999999999 or 6.00000000001

As a result, do not expect that `(x/d)*d` equals `x`!

NEVER write tests like: `if (x==6.0)` ……

Char type

type	kind	bits	maximum
char	integer	8	127
short	integer	16	32767
int	integer	32	2147483647
long	integer	64	>10E18
float	floating point	32	>10E38
double	floating point	64	>10E308

A `char` can attain values from the interval `[-128,127]`.

Type: char

The type **`char`** is used to represent characters.

E.g.: **`'0', '1', .., '9', 'a', .., 'z', 'A', .., 'Z', '#', '!'`**

```
char a, letter;
a = 'K';
letter = 'a';
```

Order: defined in the so-called ASCII table.

ASCII = American Standard Code for Information Interchange

`'0'<'1'<..<'9' < 'A'<'B'<..<'Z'  < 'a'<'b'<..<'z'`

ASCII TABLE

Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char	Decimal	Hex	Char
0	0	[NULL]	32	20	[SPACE]	64	40	@	96	60	`
1	1	[START OF HEADING]	33	21	!	65	41	A	97	61	a
2	2	[START OF TEXT]	34	22	"	66	42	B	98	62	b
3	3	[END OF TEXT]	35	23	#	67	43	C	99	63	c
4	4	[END OF TRANSMISSION]	36	24	$	68	44	D	100	64	d
5	5	[ENQUIRY]	37	25	%	69	45	E	101	65	e
6	6	[ACKNOWLEDGE]	38	26	&	70	46	F	102	66	f
7	7	[BELL]	39	27	'	71	47	G	103	67	g
8	8	[BACKSPACE]	40	28	(	72	48	H	104	68	h
9	9	[HORIZONTAL TAB]	41	29	)	73	49	I	105	69	i
10	A	[LINE FEED]	42	2A	*	74	4A	J	106	6A	j
11	B	[VERTICAL TAB]	43	2B	+	75	4B	K	107	6B	k
12	C	[FORM FEED]	44	2C	,	76	4C	L	108	6C	l
13	D	[CARRIAGE RETURN]	45	2D	-	77	4D	M	109	6D	m
14	E	[SHIFT OUT]	46	2E	.	78	4E	N	110	6E	n
15	F	[SHIFT IN]	47	2F	/	79	4F	O	111	6F	o
16	10	[DATA LINK ESCAPE]	48	30	0	80	50	P	112	70	p
17	11	[DEVICE CONTROL 1]	49	31	1	81	51	Q	113	71	q
18	12	[DEVICE CONTROL 2]	50	32	2	82	52	R	114	72	r
19	13	[DEVICE CONTROL 3]	51	33	3	83	53	S	115	73	s
20	14	[DEVICE CONTROL 4]	52	34	4	84	54	T	116	74	t
21	15	[NEGATIVE ACKNOWLEDGE]	53	35	5	85	55	U	117	75	u
22	16	[SYNCHRONOUS IDLE]	54	36	6	86	56	V	118	76	v
23	17	[END OF TRANS. BLOCK]	55	37	7	87	57	W	119	77	w
24	18	[CANCEL]	56	38	8	88	58	X	120	78	x
25	19	[END OF MEDIUM]	57	39	9	89	59	Y	121	79	y
26	1A	[SUBSTITUTE]	58	3A	:	90	5A	Z	122	7A	z
27	1B	[ESCAPE]	59	3B	;	91	5B	[	123	7B	{
28	1C	[FILE SEPARATOR]	60	3C	<	92	5C	\	124	7C	|
29	1D	[GROUP SEPARATOR]	61	3D	=	93	5D	]	125	7D	}
30	1E	[RECORD SEPARATOR]	62	3E	>	94	5E	^	126	7E	~
31	1F	[UNIT SEPARATOR]	63	3F	?	95	5F	_	127	7F	[DEL]

Single Character I/O

- `int getchar()`

  reads a single character from standard input

- `int putchar(int)`

  prints a single character on standard output

Note: yields an `int` (not a `char`)!

Example (echo character)

```
#include <stdio.h>
#include <stdlib.h>

int main(int argc, char *argv[]) {
    int c;
    c = getchar();
    putchar(c);
    putchar('\n');
    return 0;
}
```



```
printf("%c\n",c);
```

Arithmetic with characters

- The type `char` is actually an integer from [-128,127].
- For example, the character ‘A’ is (at location) 65 (in the ASCII table), and ‘B’ is (at location) 66.
- `printf ("%c", 'A'+1);`
- This prints the character at location 65+1=66 in the ASCII table. In other words, the character `'B'`.

So, this is perfectly valid:

```
char a = '1';
char b = '0';
printf ("%d\n", 10*(a - '0') + b - '0');
```

Formatted I/O: int specifiers

`%c`	character
`%d`	signed decimal
`%ld`	long decimal
`%u`	unsigned decimal

```
printf("%c %d %u\n", 'e', -17, 17);
```

Output: `e -17 17`

Formatted Output: printf

- Examples:

```
printf("%d%d", 123, 456);  // prints: 123456

printf("%d%4d", 123, 456); // prints: 123 456

printf("%04d", 123);       // prints: 0123

printf("pi=%f and e=%f.\n", 3.1415926, 2.718);

printf("%.1f\n", 1.49);    // prints 1.5
```

Formatted I/O: float specifiers

Standard precision is 11 positions (including decimal dot):

```
%f    ddd.ddd
%e    d.dddddde{sign}dd
%E    d.ddddde{sign}dd
```

```
printf("%f\n", 1234.56789);
printf("%5.3f\n", 1234.56789);
printf("%12.4e\n", 1234.56789);
```

```
1234.567890
1234.568
  1.2346e+03
```

(at least 5 positions, 3 digits after .)

(at least 12 positions, 4 digits after .)



End week 2